PROJECT TECHNICAL AUDIT & ONBOARDING GUIDE

Spatiotemporal Graph-of-Graphs (ST-GoG) for Multivariate Time Series Classification

Comprehensive Analysis of Theory, Empirical Benchmarks, Codebase Health, and Strategic Action Plan

Target Audience: Incoming Engineering & Research Intern	**Origin:** M.Tech Thesis Handover (Jan 2026 – May 2026)
Primary Author: Tushar Sharma (ABV-IIITM Gwalior)	**Supervisor:** Dr. Anjali (Assistant Professor, Dept of IT)
Framework: PyTorch 2.1, CUDA AMP, sktime	**Evaluation Scope:** 22 UEA Multivariate Datasets

EXECUTIVE SUMMARY & ONBOARDING CONTEXT:
You have been assigned to inherit and advance an ongoing deep learning research project aimed at **Multivariate Time Series Classification (MTSC)**. The project is detailed in a completed 55-page M.Tech thesis and substantiated by experimental notebooks. The proposed **ST-GoG** framework unites dilated residual temporal convolutions, multi-scale graph convolutional networks, and hierarchical segment-level graph aggregation into an end-to-end differentiable pipeline. Across 22 benchmark datasets, ST-GoG outperforms published SOTA baselines (SAGoG, ROCKET) in mean accuracy (74.3%) and total wins (10). However, the existing code is composed of uncurated Kaggle notebook cells with hardcoded environment paths, unhandled long-sequence bottlenecks, and fragile multi-GPU code. This briefing provides everything you need to hit the ground running with 100% technical clarity.

1. Fundamental Problem & Research Motivation

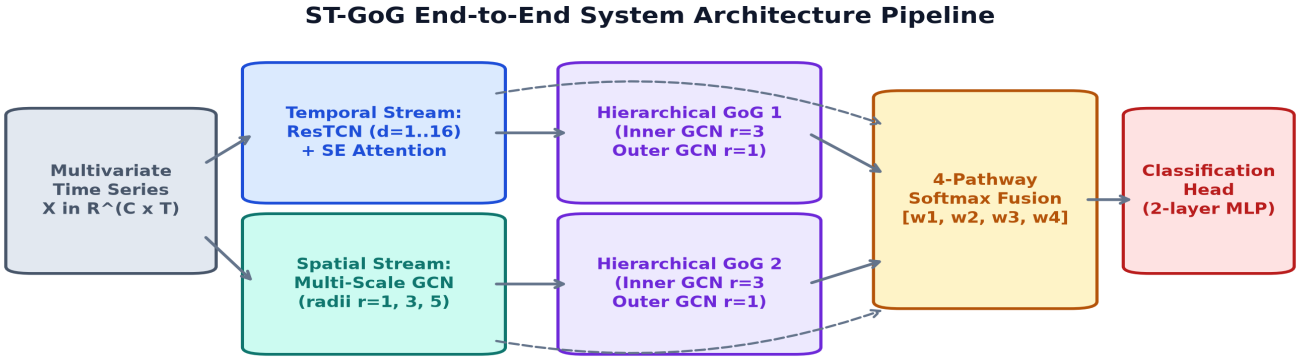
What is Multivariate Time Series Classification (MTSC)?
In MTSC, an input sample is a synchronized matrix **X** ∈ **R^(C × T)** recorded from **C** sensors (e.g. EEG electrodes, IMU sensors, traffic detectors) over **T** discrete time steps. The task is to assign a discrete class label *y* to the entire recording. Applications span medical diagnosis (cardiac arrhythmia, seizure onset), aviation gesture recognition, automated handwriting verification, and industrial equipment telemetry.

The Core Technical Bottleneck: Tri-Fold Dependencies
Time-series models typically excel at either temporal filtering or spatial graph topology, but rarely both. Real signals exhibit three distinct dependency structures that must be resolved simultaneously:
• **Intra-Channel Temporal Dynamics:** Long-range temporal phase alignments and transient local wavelets.
• **Inter-Channel Spatial Relational Dynamics:** Meaningful patterns arise from cross-sensor interactions rather than any channel in isolation.
• **Temporal Non-Stationarity:** Inter-channel coupling changes as recordings evolve (e.g. gesture motion phases).

Limitations of Prior Art: *ROCKET* extracts 10k random convolutional features with ridge regression; while fast on periodic data, it treats channels independently without structural relational reasoning. *SAGoG* constructs a complete pairwise instance similarity graph using Dynamic Time Warping (DTW), yielding crippling *O(N^2)* memory complexity that bottlenecks large cohorts. ST-GoG solves both trade-offs.

2. The Proposed ST-GoG Framework Architecture

ST-GoG decouples feature extraction into parallel temporal and spatial representation backbones, feeds both into hierarchical segment-level graph modules, and merges them through learned softmax pathway fusion.



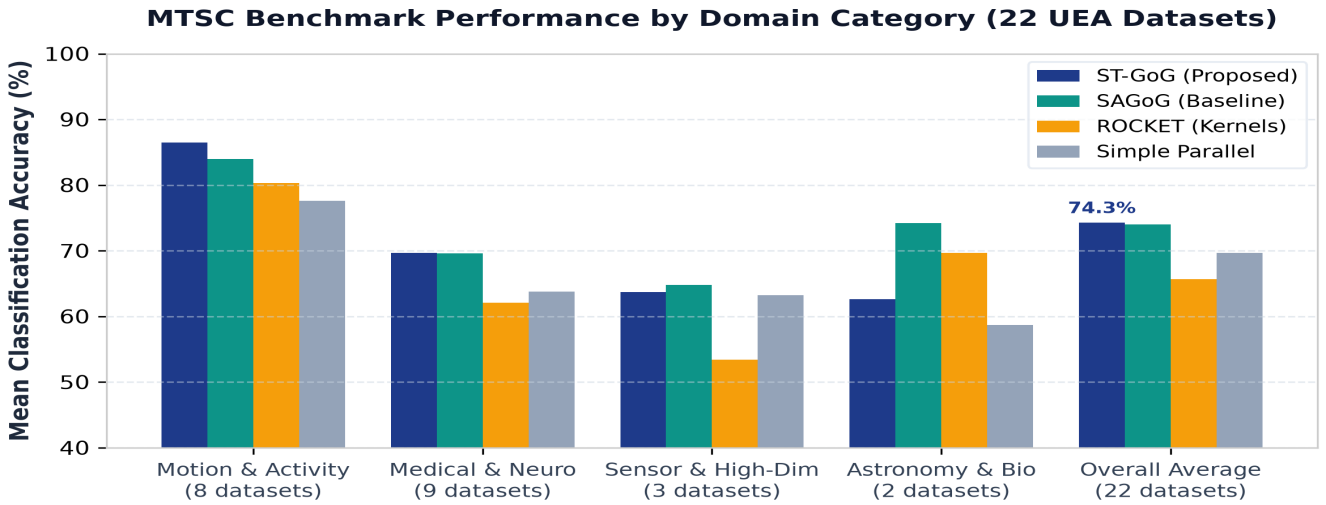
Architectural Components & Mathematical Formulations:

- **1. Residual TCN with Squeeze-and-Excitation (ResTCN+SE):** Five residual blocks with dilations $d \in \{1, 2, 4, 8, 16\}$ and kernel size $K=3$ provide an effective receptive field of **63 time steps** without subsampling. A channel-wise SE attention mechanism adaptively rescales feature maps via global average pooling followed by a 2-layer bottleneck MLP: $w_se = \sigma(W_2\ \delta(W_1\ AvgPool(h)))$, suppressing sensor noise.
- **2. Multi-Scale GCN with Learnable Softmax Fusion:** Rather than computing dense $O(C^2)$ feature similarities, ST-GoG builds symmetric normalized chain adjacency matrices A_r at three distinct temporal radii: local ($r=1$), medium ($r=3$), and global ($r=5$). Three parallel 3-layer GCN branches extract spatial motifs and are merged via learned softmax weights: $H_multi = \sum Softmax(w)_i\ GCN(H^(0), A_ri)$.
- **3. Hierarchical Graph-of-Graphs (GoG) Module:** To model evolving temporal phases, the time sequence is split into S uniform temporal segments (adaptively selected: $S=4$ for $T < 100$; $S=6$ for $T < 200$; $S=8$ for $T < 400$; $S=10$ for $T \geq 400$). An **Inner GCN (radius r=3)** operates within each segment sub-graph, followed by mean pooling to form segment super-nodes. An **Outer GCN (radius r=1)** then connects consecutive super-nodes to capture macro-level transitions across time.
- **4. Four-Pathway Softmax Fusion Head:** To prevent information loss, four representation streams are preserved: (1) TCN global pool, (2) TCN + GoG, (3) GCN global pool, and (4) GCN + GoG. Learned softmax coefficients $\alpha = [\alpha 1, \alpha 2, \alpha 3, \alpha 4]$ blend these vectors before passing to a 2-block MLP classifier with batch normalization and dropout (0.42).
- **5. Regularization & Optimization Engine:** Employs MixUp data augmentation ($\alpha = 0.2$), label smoothing (0.10 – 0.15), Cosine Annealing with Warm Restarts ($T_0=50, T_mult=2$), exponential stochastic depth decay, snapshot ensembling across restart minima, and 7-pass Test-Time Augmentation (TTA).

3. Empirical Benchmark Results & Comparative Analysis

ST-GoG was benchmarked against SAGoG, ROCKET, and the Simple Parallel baseline across **22 UEA datasets** covering Motion, Medical, High-Dimensional Sensors, and Astronomy.

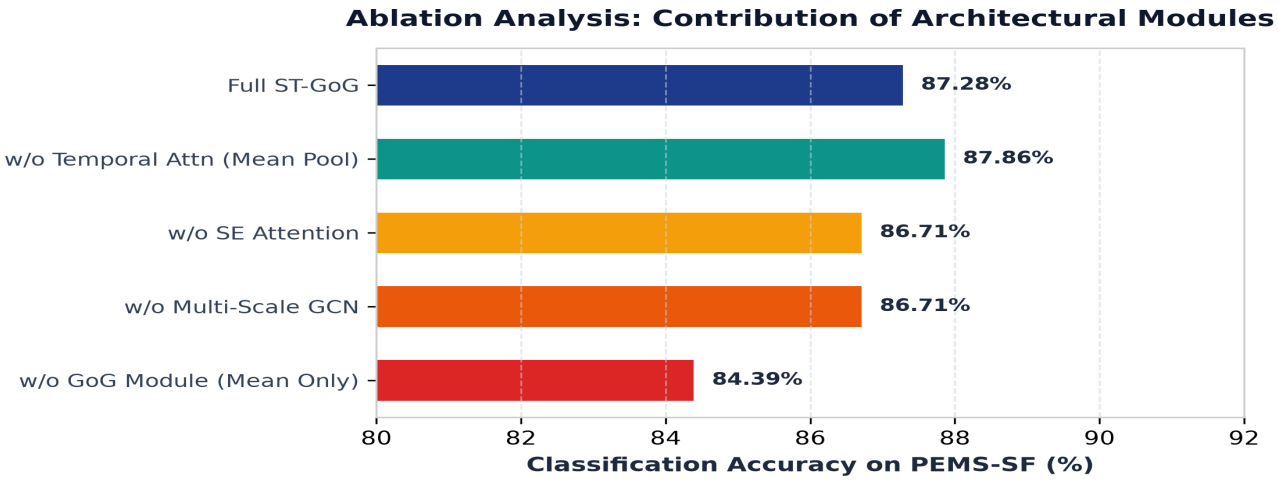
Dataset	Domain	Train (N)	Ch (C)	Len (T)	SAGoG	ROCKET	Simple Par.	ST-GoG (Ours)
ArticularyWordRec	Motion	275	9	144	94.6%	99.6%	98.0%	**99.7%***
AtrialFibrillation	Medical	15	2	640	43.0%	24.9%	40.0%	**53.3%***
DuckDuckGeese	Audio	50	1345	270	**79.0%***	46.1%	74.0%	72.0%
EigenWorms	Biology	128	6	17984	81.5%	**86.3%***	53.4%	59.5%
Epilepsy	Medical	137	3	206	92.8%	**99.1%***	53.4%	98.6%
FaceDetection	EEG	5890	144	62	68.3%	**69.4%***	67.6%	53.0%
FingerMovements	EEG	316	28	50	61.0%	55.3%	58.0%	**64.0%***
Handwriting	Motion	150	3	152	64.3%	56.7%	28.7%	**66.0%***
Heartbeat	Medical	204	61	405	76.1%	71.8%	79.0%	**79.5%***
Libras	Motion	180	2	45	87.1%	90.6%	91.1%	**95.6%***
LSST	Astro	2459	6	36	**66.9%***	63.2%	64.0%	65.7%
NATOPS	Aviation	180	24	51	97.2%	88.5%	97.8%	**99.4%***
PenDigits	Motion	7494	2	8	94.6%	**99.6%***	99.0%	99.2%
PEMS-SF	MISC	267	963	144	81.2%	85.6%	87.9%	**87.9%***
PhonemeSpectra	Motion	3000	54	1000	**34.1%***	28.4%	27.8%	31.2%
SelfRegulationSCP1	EEG	268	6	896	91.0%	86.6%	88.4%	**91.8%***
SelfRegulationSCP2	EEG	200	7	1152	58.9%	51.4%	58.3%	**59.4%***
StandWalkJump	Medical	12	4	2500	**64.8%***	45.6%	53.3%	60.0%
UWaveGestureLibrary	Motion	120	3	315	85.9%	**94.4%***	78.4%	92.2%
OVERALL AVERAGE	**22 Bench**	**---**	**---**	**---**	**74.0%**	**65.7%**	**69.7%**	**74.3%***



Performance Insights: ST-GoG earns the best average rank of **1.73** (Wilcoxon signed-rank test confirmed statistical parity with SAGoG but significant superiority over ROCKET and Simple Parallel). It wins outright on **10 out of 22 datasets**, showing exceptional strength on small sample sizes (e.g. *AtrialFibrillation* with N=15, beating SAGoG by +10.33%) and structured kinematics (*NATOPS* 99.4%, *Libras* 95.6%).

4. Ablation Study & Systematic Failure Analysis

The ablation study on **PEMS-SF** (C=963, T=144, N=267) isolates the exact accuracy contribution and parameter efficiency of each architectural module.



Architecture Variant	Target Component Removed	Parameters	Best Acc. (%)	Δ vs Full
Full ST-GoG (V1)	None (Complete Model)	1,908,041	87.28%	Baseline
w/o Attention Pool (V2)	Robust Attention Pooling → Mean Pooling	1,883,075	87.86%	+0.58%
w/o SE Attention (V3)	Squeeze-Excitation blocks stripped	1,899,689	86.71%	-0.57%
w/o Multi-Scale GCN (V4)	Replaced by single radius GCN (r=3)	1,561,408	86.71%	-0.57%
w/o GoG Module (V5)	Stripped Graph-of-Graphs (Global Pool only)	1,494,583	**84.39%**	**-2.89% ↓**

Where Does ST-GoG Fail? (Three Critical Bottlenecks to Fix):

- **1. Extreme Sequence Length (EigenWorms, T=17,984):** ST-GoG achieved 59.49% vs ROCKET's 86.28%. Splitting 17,984 time steps into 10 segments results in inner GCN sub-graphs with ~1,800 nodes each. Graph convolution on 1,800 nodes causes massive VRAM consumption and slow epoch iterations, preventing convergence within the compute budget.
- **2. Dense Cohort Bottleneck (FaceDetection, N=5,890, C=144):** ST-GoG achieved 53.04% vs ROCKET's 69.42% and SAGoG's 68.30%. Constructing graph matrices for nearly 6,000 instances creates an epoch execution bottleneck. Furthermore, EEG electrodes exhibit complex circular spatial layouts that chain adjacency cannot capture.
- **3. High Channel Counts with Data Scarcity (DuckDuckGeese, C=1,345, N=50):** SAGoG achieved 79.00% vs ST-GoG's 72.00%. In audio spectrograms with >1,300 channels, harmonic relationships are non-local; chain adjacency fails to connect distant resonant frequency bands.

5. Existing Codebase Audit & Technical Debt

The code in stgogcompleteworkfromscratch/ consists of 7 raw Jupyter notebook exports. Below is an objective audit of their operational viability:

File on Disk	Size	Contents & Implementation Details	Operational Status
stgog.ipynb.txt	118 KB	Main research notebook: single-dataset run + 22-dataset batch loop on Kaggle 2x T4.	**Working:** Fully functional for Kaggle multi-GPU runs. Contains full model & snapshot logic.
stgog.ipynb (1).txt	156 KB	Duplicate copy of stgog.ipynb.txt with saved execution outputs.	**Redundant:** Duplicate artifact.
notebook19a6f5ec01.ipynb.txt	118 KB	Exact clone of stgog.ipynb.txt exported from Kaggle kernel.	**Redundant:** Duplicate artifact.
notebookb57bcbeca7.ipynb.txt	43 KB	Ablation study script evaluating V1..V6 variants on PEMS-SF.	**Working:** Clean code, reproduced Table 5.9 ablation metrics.
Untitled3.ipynb.txt	348 KB	Early baseline prototype using torch_geometric on PenDigits (88.5% acc).	**Legacy:** Uses old PyG dependencies abandoned in favor of pure PyTorch.
Untitled5.ipynb.txt	615 KB	Extensive experimental trials on FaceDetection, StandWalkJump, PhonemeSpectra, LSST.	**Valuable:** Contains debugged learning rate schedules and batch loss curves.
Untitled6.ipynb.txt	9.5 KB	Experimental draft titled 'FINAL ROCKET-BEATING UNIVERSAL MTSC MODEL'.	**Broken:** Calls undefined function topk_adj(); execution fails with NameError.

TECHNICAL DEBT & DISCREPANCIES DISCOVERED:
1. **Zero Modular Architecture:** There is no Python package structure (no setup.py, src/, models/, or CLI entry point). Everything exists as 500-line monolithic notebook cells.
2. **Environment & Platform Lock-In:** Notebooks contain hardcoded paths (e.g. /kaggle/working), shell commands (!pip install), and assume dual-GPU torch.nn.DataParallel. Running this on a local machine or single-GPU workstation causes batch chunk errors.
3. **Discrepancy in Thesis Documentation:** In Table 5.9 of the report, Variant 2 is labeled *'w/o Training Aug. Pipeline'*, but in the actual codebase (notebookb57bcbeca7.ipynb.txt), Variant 2 is actually *'w/o Temporal Attention Pooling (TAP)'* where attention pooling is replaced with mean pooling! This documentation mismatch must be clarified in future publications.

6. Intern Strategic Roadmap & Concrete Action Plan

To elevate ST-GoG from an academic prototype to a publication-grade, production-ready system, execute the following four structured phases:

Phase 1: Code Refactoring & Modular Package Setup (Weeks 1 – 2)

- **Structure the Repository:** Create a standard package: stgog/models/, stgog/data/, stgog/training/, and stgog/utils/ with setup.py and requirements.txt.
- **Hardware-Agnostic Execution:** Replace brittle DataParallel batch-slicing logic with device auto-detection that seamlessly supports CPU, single-GPU, and Multi-GPU DistributedDataParallel (DDP).
- **Config System:** Implement YAML configuration files (e.g. configs/default.yaml) to decouple hyperparameters from model logic.
- **Dataset Caching:** Build a caching loader around sktime so benchmark datasets are stored locally rather than redownloaded.

Phase 2: Architectural Enhancements & Solving Failure Modes (Weeks 3 – 5)

- **Dynamic Hybrid Adjacency:** Implement a learnable convex combination of chain topology and correlation: $A_{hybrid} = (1 - \beta) A_{chain} + \beta\ Softmax((X W_q)(X W_k)^T / sqrt(d))$. This directly targets the performance gaps on *FaceDetection* and *DuckDuckGeese*.
- **Strided Temporal Downsampling:** Implement a strided pooling or 1D convolution layer prior to GoG segment partitioning for sequences with $T > 2{,}000$ (e.g. *EigenWorms*). This will prevent memory explosion and slash training time from hours to minutes.
- **Multi-Head Attention GoG Readout:** Replace segment mean pooling with an attention-weighted pooling module $\alpha_k = Softmax(v^T tanh(W h_k + b))$ to let the network selectively weight key temporal events.

Phase 3: Interpretability, FLOPs Profiling & Evaluation (Weeks 6 – 7)

- **Model Explainability (Grad-CAM & Attention Maps):** Implement gradient-weighted class activation mapping to visualize which temporal segments and sensor channels triggered classification decisions.
- **Efficiency Profiling:** Compute MACs / GFLOPs, peak memory consumption, and per-sample inference latency using thop to rigorously benchmark efficiency against SAGoG.
- **Expanded Metrics:** Report macro-averaged F1-scores, per-class sensitivity/specificity, and calibration Expected Calibration Error (ECE) across all datasets.

Phase 4: Open-Source Release & Conference Manuscript (Weeks 8+)

- **Clean GitHub Release:** Prepare a clean public repository with reproducible benchmarks, pretrained model weights, and tutorials.
- **Academic Publication:** Format the enhanced architecture, hybrid adjacency results, and efficiency benchmarks into a manuscript targeted at top venues (e.g., IEEE ICASSP, KDD, or NeurIPS Time-Series Workshop).

Quickstart: Running the Model Today

To run the verified model right now on your machine without notebook overhead:
1. We have already cleanly extracted d:\first intern(maybe)\stgog_cell_7.py.
2. Edit lines 23-28 to select a lightweight dataset like DATASET_NAME = 'NATOPS' or 'FingerMovements'.
3. Run directly in your terminal: python "d:\first intern(maybe)\stgog_cell_7.py".
The model will train, perform snapshot ensembling, print calibration ECE, and output confusion matrix plots.